```
Aloita koodaus tai luo tekoälyllä.
```

Statistical Methods in ML II: Assignment 2

- **Deadline**: 11:59PM February 18th
- **Submission**: You need to submit your solutions through Crowdmark, including all your derivations, plots, and your code. You can produce the files however you like (e.g. $\LaTeX$, Microsoft Word, etc), as long as it is readable. Points will be deducted if we have a hard time reading your solutions or understanding the structure of your code.
- **Collaboration policy**: After attempting the problems on an individual basis, you may discuss and work together on the assignment with up to two classmates. However, **you must write your own code and write up your own solutions individually and explicitly name any collaborators** at the top of the homework.

Q1 - Image Denoising

In this problem, we will implement the sum-product Loopy **belief propagation** (Loopy-BP) method for denoising binary images which you have seen in tutorial 4. We will consider images as matrices of size $\sqrt{n} \times \sqrt{n}$. Each element of the matrix can be either 1 or -1, with 1 representing white pixels and -1 representing black pixels. This is different from the $0/1$ representation commonly used for other CV tasks. This notation will be more convenient when multiplying with pixel values.

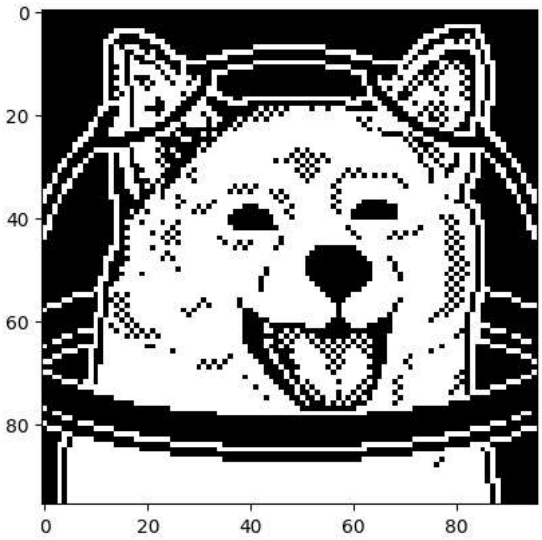
Data preparation

Below we provide you with code for loading and preparing the image data.

First, we load a black and white image of Laika and convert it into a binary matrix of 1 and -1. So that white pixels have value 1 and black pixels have value -1.

```
!pip install wget

import numpy as np
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
import PIL.Image as Image
from os.path import exists
from wget import download
from tqdm import tqdm
filename, url = "3vaef0cog4f61.png", "https://i.redd.it/3vaef0cog4f61.png"

def load_img():
    if not exists(filename):
        download(url)

    with open(filename, 'rb') as fp:
        img2 = Image.open(fp).convert('L')
        img2 = np.array(img2)
    return (img2[:96,11:107] > 120) * 2.0 - 1

img_true = load_img()
plt.imshow(img_true, cmap='gray')
```

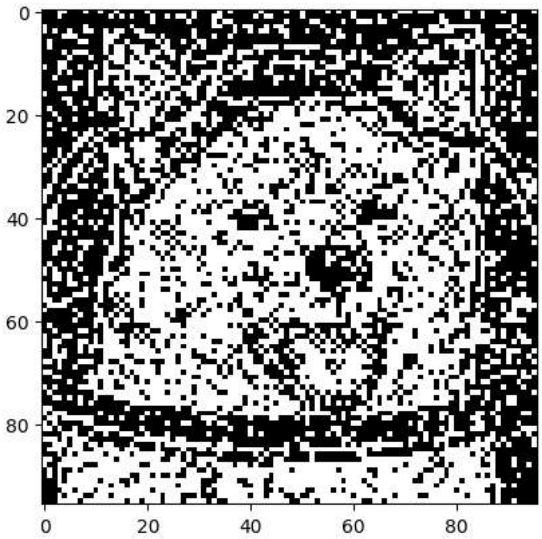
```
Requirement already satisfied: wget in /usr/local/lib/python3.10/dist-packages (3.2)
<matplotlib.image.AxesImage at 0x7c67c811a050>
```



To introduce noise into the image, for each pixel, swap its value between 1 and -1 with rate 0.2.

```
def gen_noisyimg(img, noise=.05):
    swap = np.random.binomial(1, noise, size=img.shape)
    return img * (2 * swap - 1)

noise = 0.2
img_noisy = gen_noisyimg(img_true, noise)
plt.imshow(-1 * img_noisy, cmap='gray')
```

```
<matplotlib.image.AxesImage at 0x7c67c81c50c0>
```



The Loopy BP algorithm

Recall from lecture and tutorial, the Loopy-BP algorithm iteratively updates the messages of each node through a sum-product operation. The **sum-product** operation computes the joint inbound message through multiplication, and then marginalizes the factors through summation. This is in contrast to the **max-product** BP, which computes the maximum a-posteriori value for each variable through taking the maximum over variables.

Initialization:

For discrete node x_j with 2 possible states, $m_{i\to j}$ can be written as a 2 dimensional real vector $\mathrm{m}_{i,j}$ with $m_{i\to j}(x_j) = \mathrm{m}_{i,j}[index(x_j)]$. We initialize them uniformly to $m_{i\to j}(x_j) = 1/2$.

(Aside: for continuous cases, $m_{i\to j}(x_j)$ is a real valued function of x_j. We only need to deal with the discrete case here.)

For a number of iterations:

For node x_j in $\{x_s\}_{s=1}^n$:

1. Compute the product of inbound messages from neighbours of x_j:
$$\prod_{k \in N(j) \neq i} m_{k \to j}(x_j)$$
2. Compute potentials $\psi_j(x_j) = \exp(\beta x_j y_j)$ and $\psi_{ij}(x_i, x_j) = \exp(J x_i x_j)$. This expression specifically holds when $x \in \{-1, +1\}$.
3. Marginalize over $x_j = \{-1, +1\}$ to get $m_{j \to i}(x_i)$:
$$m_{j \to i}(x_i) = \sum_{x_j} \psi_j(x_j) \psi_{ij}(x_i, x_j) \prod_{k \in N(j) \neq i} m_{k \to j}(x_j)$$
4. Normalize messages for stability $m_{j \to i}(x_i) = m_{j \to i}(x_i) / \sum_{x_i} m_{j \to i}(x_i)$.

Compute beliefs after message passing is done.
$$b(x_i) \propto \psi_i(x_i) \prod_{j \in \mathcal{N}(i)} m_{j \to i}(x_i).$$

You'll be tasked to perform steps 1-3 in the iterations and computing the beliefs. We will provide you with helper functions for initialization, finding neighbours, and normalization.

Initialization

Initialize the message between neighbor pixels uniformly as $m_{j \to i}(x_i) = 1/k$. Since each pixel can only be 1 or -1, message has two values $m_{j \to i}(1)$ and $m_{j \to i}(-1)$. We also initialize hyperparameters J and β.

```
y = img_noisy.reshape([img_true.size, ])
num_nodes = len(y)
init_message = np.zeros([2, num_nodes, num_nodes]) + .5
J = 1.0
beta = 1.0
```

Find the neighboring pixels around a given pixel, which will be used for BP updates

```
def get_neighbors_of(node):
    """
    arguments:
     int node:  in [0,num_nodes) index of node to query
    globals:
     int num_nodes: number of nodes
    return: set(int) indices of neighbors of queried node
    """
    neighbors = []
    m = int(np.sqrt(num_nodes))
    if (node + 1) % m != 0:
        neighbors += [node + 1]
    if node % m != 0:
        neighbors += [node - 1]
    if node + m < num_nodes:
        neighbors += [node + m]
    if node - m >= 0:
        neighbors += [node - m]

    return set(neighbors)
```

Q1.1 Implement message passing in BP (20 points)

Implement the function `get_message()` that computes the message passed from node j to node i:
$$m_{j \to i}(x_i) = \sum_{x_j} \psi_j(x_j) \psi_{ij}(x_i, x_j) \prod_{k \in N(j) \neq i} m_{k \to j}(x_j)$$
`get_message()` will be used by (provided below) `step_bp()` to perform one iteration of loopy-BP: it first normalizes the returned message from `get_message()`, and then updates the message with momentum `1.0 - step`.

```
def get_message(node_from, node_to, messages):
    """
    arguments:
     int node_from: in [0,num_nodes) index of source node
     int node_from: in [0,num_nodes) index of target node
     float array messages: (2, num_nodes, num_nodes), messages[:,j,i] is message
                           from node j to node i
    reads globals:
     float array y: (num_nodes,) observed pixel values
```

```
         float J: clique coupling strength constant
         float beta: observation to true pixel coupling strength constant
        return: array(float) of shape (2,) un-normalized message from node_from to
        node_to
        """
        #TODO: implement your function here
        pass


    def step_bp(step, messages):
        """
        arguments:
         float step: step size to update messages
        return
         float array messages: (2, num_nodes, num_nodes), messages[:,j,i] is message
                               from node j to node i
        """
        for node_from in range(num_nodes):
            for node_to in get_neighbors_of(node_from):
                m_new = get_message(node_from, node_to, messages)
                # normalize
                m_new = m_new / np.sum(m_new)

                messages[:, node_from, node_to] = step * m_new + (1. - step) * \
                    messages[:, node_from, node_to]
        return messages
```

Then, run loopy BP update for 10 iterations:

```
num_iter = 10
step = 0.5
for it in range(num_iter):
    init_message = step_bp(step, init_message)
    print(it + 1,'/',num_iter)
```

Q1.2 Computing belief from messages (10 points)

↳ 4 solua piilotettu

Question 1.3 Momentum in belief propagation

In the sample code provided above, we performed message update with a momentum parameter `step`. In this question, you will experimentally investigate how momentum affects the characteristics of convergence.

Question 1.3.1 (5 points)

Complete the function `test_trajectory` below to obtain predicted image after each step of message passing. Return predicted images as list.

↳ 1 solu piilotettu

Question 1.3.2 (5 points)

Use test trajectory to create image serieses for `step size` 0.1, 0.3, and 1.0, each with 10 steps. Display these images with `plot_series' provided below.

In the textbox below, 1. comment on what happens when a large step size is used for too many iterations. 2. how would you adjust other hyperparameters to counteract this effect?

↳ 1 solu piilotettu

Response to 1.3.2 (Enter your response below):

Question 1.4 Noise level and the hyperparameter J

In this question, we will study how the level of noise in the image influences our choice in the hyperparameter J.

Question 1.4.1 (5 points)

First, generate and display images with noise of 0.05, 0.3. In the text box below, comment on what would happen if noise was set to 0.5 and 1.0

```
# Solution
```

Response to 1.4.1 (enter your response below):

Question 1.4.2 (5 points)

Now, perform image denoising on images with noise levels 0.05 and 0.3 using using $J = 0.1$, $J = 0.5$, $J = 1.0$, and $J = 5.0$. Set step size to 0.8 and max_step to 5. Plot the denoised images (if reusing `test_trajectory`, you should plot 8 image serieses). In text box below, comment on what you observe and provide a brief explanation on why this might occur.

```
# Solution
```

Response to 1.4.2 (enter your response below):

Question 2: Markov chain Monte Carlo in the TrueSkill model

The goal of this question is to get you familiar with the basics of Bayesian inference in medium-sized models with continuous latent variables, and the basics of Hamiltonian Monte Carlo.

Background

We'll implement a variant of the TrueSkill model, a player ranking system for competitive games originally developed for Halo 2. It is a generalization of the Elo rating system in Chess.

This assignment is based on this one developed by Carl Rasmussen at Cambridge for his course on probabilistic machine learning.

Model definition

We'll consider a slightly simplified version of the original trueskill model. We assume that each player has a true, but unknown skill $z_i \in \mathbb{R}$. We use N to denote the number of players.

The prior:

The prior over each player's skill is a standard normal distribution, and all player's skills are *a priori* independent.

The likelihood:

For each observed game, the probability that player i beats player j, given the player's skills z_A and z_B, is:

$$p(A \text{ beat } B | z_A, z_B) = \sigma(z_A - z_B)$$

where

$$\sigma(y) = \frac{1}{1 + \exp(-y)}$$

We chose this function simply because it's close to zero or one when the player's skills are very different, and equals one-half when the player skills are the same. This likelihood function is the only thing that gives meaning to the latent skill variables $z_1 \ldots z_N$.

There can be more than one game played between a pair of players. The outcome of each game is independent given the players' skills. We use M to denote the number of games.

```
!pip install wget
import os
import os.path

import matplotlib.pyplot as plt
import wget

import pandas as pd


import numpy as np
from scipy.stats import norm
```

```
import scipy.io
import scipy.stats
import torch
import random
from torch.distributions.normal import Normal

from functools import partial

import matplotlib.pyplot as plt
```

Q 2.1 Implementing the TrueSkill Model [10 points]

Q 2.1.a [4 points]

Implement a function **`log_joint_prior`** that computes the log of the prior, jointly evaluated over all player's skills.

Specifically, given a $K \times N$ array where each row is a setting of the skills for all N players, it returns a $K \times 1$ array, where each row contains a scalar giving the log-prior for that set of skills.

```
def log_joint_prior(zs_array):
  # TODO
  pass
```

Q 2.1.b [6 points]

Implement two functions **`logp_a_beats_b`** and **`logp_b_beats_a`**.

Given a pair of skills z_a and z_b, **`logp_a_beats_b`** evaluates the log-likelihood that player with skill z_a beat player with skill z_b under the model detailed above, and **`logp_b_beats_a`** is vice versa.

To ensure numerical stability, use the function $torch.logaddexp$

```
def logp_a_beats_b(z_a, z_b):
  # Hint: Use torch.logaddexp
  # TODO
  pass

def logp_b_beats_a(z_a, z_b):
  # Hint: Use torch.logaddexp
  # TODO
  pass
```

Q 2.2 Examining the posterior for only two players and toy data [10 points]

To get a feel for this model, we'll first consider the case where we only have 2 players, A and B. We'll examine how the prior and likelihood interact when conditioning on different sets of games.

Provided in the starter code is a function **`plot_isocontours`** which evaluates a provided function on a grid of z_A and z_B's and plots the isocontours of that function. There is also a function **`plot_2d_fun`**. We have included an example for how you can use these functions.

↳ 10 solua piilotettu

Q 2.3 Hamiltonian Monte Carlo on Two Players and Toy Data [4 points]

One nice thing about a Bayesian approach is that it separates the model specification from the approximate inference strategy. The original Trueskill paper from 2007 used message passing. Carl Rasmussen's assignment uses Gibbs sampling.

In this question, we will approximate posterior distributions with gradient-based Hamiltonian Monte Carlo.

In the next assignment, we'll use gradient-based stochastic variational inference, which wasn't invented until around 2014.

↳ 8 solua piilotettu

Q 2.4 Approximate inference conditioned on real data [26 points]

The dataset contains data on 2546 chess games amongst 1434 players:

- names is a 1163 by 1 matrix, whose i'th entry is the name of player i.

- games is a 2543 by 2 matrix of game outcomes (actually chess matches), one row per game.

The first column contains the indices of the players who won. The second column contains the indices of the player who lost.

It is based on the kaggle chess dataset: https://www.kaggle.com/datasets/datasnaek/chess